Rapport intermédiaire

Réseau virtuel de machines en Bash

LO14 - UTT - 2026-2027

Jules et Selyan - 8 novembre 2026

Version de travail à relire par le binôme avant remise. Référence technique : `9a56994`.

1. Objectif et périmètre

Le projet consiste à développer la commande Bash `rvsh`. Elle simule un réseau de machines Linux : les machines, utilisateurs et droits sont des données du programme, consultées depuis plusieurs terminaux sur le même hôte. Le programme ne lance pas de machine virtuelle système et ne crée pas de compte Linux.

Deux modes structurent l'interface. `rvsh -admin` authentifie l'administrateur et ouvre le prompt `root@hostroot >`. `rvsh -connect machine utilisateur` vérifie la machine, le compte, son droit d'accès et son mot de passe avant d'ouvrir une session utilisateur. La machine `hostroot` et le compte root existent après l'initialisation du réseau.

Le sujet impose des commandes d'administration, de consultation des sessions, de changement d'identité, de navigation entre machines et de messagerie. Une attention particulière concerne les connexions simultanées du même compte et le retour à la machine précédente avec `exit`.

2. Structure du programme

Le point d'entrée `rvsh` charge trois modules Bash. Le découpage distingue la gestion des données, celle des sessions et l'interprétation des commandes virtuelles.

Fichier	Responsabilité
`rvsh`	Arguments de lancement, authentification initiale et choix du mode.
`lib/storage.sh`	Données partagées, verrouillage, mots de passe, droits et contrôle des processus.
`lib/session.sh`	Connexions imbriquées, identité courante, notifications et nettoyage.
`lib/commands.sh`	Commandes disponibles et contrôle des privilèges administrateur.
`tests/`	Contrat de lancement et tests fonctionnels à plusieurs terminaux.

Le shell n'évalue jamais la saisie comme du code Bash. Une commande inconnue est refusée. Les noms sont des identifiants simples ; les messages conservent les espaces. Ce choix limite les ambiguïtés de parsing pour le périmètre demandé.

3. Données persistantes et accès concurrents

Un dossier `data/`, absent de Git, contient les machines, les comptes, les autorisations, les informations complémentaires, les sessions et les messages. La variable `RVSH_DATA_DIR` permet de choisir un autre réseau indépendant. Tous les terminaux d'un réseau doivent pointer vers le même dossier.

Donnée	Représentation
Machines	Un dossier par nom sous `hosts/`.
Utilisateurs	Un dossier sous `users/`, avec mot de passe haché, droits et informations.
Sessions	Un enregistrement par connexion sous `sessions/`.
Messages immédiats	Une boîte de réception par session sous `inbox/`.
Messages différés	Une boîte par utilisateur sous `mail/`.

Les opérations partagées utilisent un verrou exclusif `flock`. Les lectures qui doivent rester cohérentes et les modifications de plusieurs fichiers s'effectuent sous ce verrou. La saisie d'un mot de passe se fait hors du verrou pour ne pas immobiliser le réseau pendant une interaction humaine. Après la saisie, l'état est revérifié avant l'application du changement.

Les fichiers du mot de passe sont remplacés par renommage d'un fichier temporaire situé dans le même dossier. Les permissions prévues sont 700 pour le dossier de données et 600 pour les fichiers créés. Le programme refuse un dossier non vide qui ne possède pas son marqueur de données, afin de ne pas modifier un dossier étranger au projet.

4. Authentification et autorisations

Au premier lancement administrateur, le programme demande de choisir puis confirmer le mot de passe de root. Aucun mot de passe prédéfini n'est versionné. OpenSSL calcule un hachage SHA-512 crypt avec un sel aléatoire ; le mot de passe n'est pas affiché pendant la saisie.

Un compte ordinaire ne reçoit aucun droit implicite lors de sa création. L'administrateur attribue les machines autorisées avec `user grant`. Les commandes de gestion exigent l'identité root dans une session issue du mode admin. Le compte root et la machine `hostroot` sont protégés contre la suppression.

Le stockage étant accessible au compte Linux qui exécute les terminaux, cette authentification appartient à la simulation. Elle ne constitue pas une séparation de sécurité entre utilisateurs du système d'exploitation.

5. Sessions et messages

Les connexions imbriquées utilisent une pile. Un parcours machineA, machineB puis machineC ajoute trois connexions distinctes. Chaque `exit` retire la connexion au sommet et restaure la précédente. Les connexions précédentes restent recensées comme connectées jusqu'à leur fermeture. Ce comportement explicite un point que le sujet ne détaille pas entièrement.

Chaque session contient l'utilisateur, la machine, la date de connexion, le PID et une identité du processus composée du démarrage du système et du démarrage du processus. Cette identité évite de conserver une ancienne session lorsque Linux réattribue son PID à un autre processus.

`write` dépose un message dans les boîtes des sessions correspondant au couple utilisateur-machine. `wall` cible toutes les sessions connectées. `wall -n` ajoute une livraison différée pour les comptes entièrement hors ligne. Le message différé est consommé à la première reconnexion du compte. Un processus de notification consulte les boîtes environ toutes les 0,2 seconde.

6. État au 8 novembre et validation prévue

La structure décrite dispose déjà d'une implémentation et de tests automatisés. Cette note expose les choix techniques réellement présents dans le code ; elle ne prétend pas retracer une planification antérieure à l'implémentation.

La validation actuelle comporte 11 vérifications du lancement et 10 scénarios d'intégration sous Linux. Ils couvrent notamment les droits, les connexions imbriquées, les mots de passe, les sessions multiples, les messages différés, les opérations concurrentes et la fermeture des processus. ShellCheck ne signale aucun diagnostic.

Avant la remise, le binôme doit relire les choix, exécuter la démonstration sur son environnement Linux/WSL et confirmer que chacun peut expliquer les modules et leurs interfaces. Le rapport final pourra s'appuyer sur les résultats de cette recette humaine en complément des preuves automatisées.

7. Livrables et échéances

Le premier rapport doit décrire la structure envisagée et les choix particuliers avant le 4 décembre 2026. Le rapport final est demandé avant le 10 janvier 2027. Le sujet annonce une présentation et un test d'environ vingt minutes pendant la première semaine de janvier ; le créneau exact reste à confirmer.

Références : sujet LO14 2026-2027 fourni dans le dépôt, code `9a56994`, README et `docs/VALIDATION.md`. La répartition personnelle des contributions doit être décrite par le binôme si elle est demandée, sans déduire cette répartition du seul nombre de commits.